Documento de apoyo (Sesión 4, Unidad 2)

Orquestación de pipelines de datos con Apache Airflow 3.x

Teoría práctica para entender y usar Airflow en flujos ETL/ELT

Javier Andrés Valdez González / Auxiliar

Seminario de Sistemas 2 — Sección P

Universidad de San Carlos de Guatemala

Febrero de 2026

Resumen

Este documento complementa el material de la Sesión 3 (ETL y su rol en OLAP) enfocándose en el *cómo operar* un flujo de datos de forma confiable: **orquestación**. Apache Airflow 3.x es un orquestador de workflows que permite describir procesos como un grafo acíclico dirigido (DAG), ejecutar tareas con control de dependencias, reintentos, observabilidad y gobierno operativo. El énfasis aquí no es redefinir ETL, sino explicar los conceptos, componentes y patrones necesarios para **autoría, ejecución y operación** de DAGs en Airflow 3.x, considerando los cambios arquitectónicos y de interfaz pública introducidos en la versión 3.

1. Introducción: de “transformar datos” a “operar procesos”

En un curso de inteligencia de negocios, la construcción de un pipeline rara vez falla por la idea conceptual (extraer–transformar–cargar), sino por su operación diaria: dependencias entre pasos, ejecuciones parciales, reintentos, *backfills*, credenciales, auditoría, trazabilidad, alertas y recuperación ante errores. Airflow se sitúa en ese plano operativo: define **qué** se ejecuta, **cuándo** se ejecuta y **bajo qué condiciones** se considera exitoso.

Airflow 3.0 marca un punto de inflexión al adoptar una arquitectura *service-oriented* y una **interfaz pública estable** para autores de DAGs basada en `airflow.sdk` [5], [6]. Por tanto, aprender Airflow 3.x implica dominar: (i) el modelo mental DAG–Task–Run, (ii) la planificación (scheduling) y control de concurrencia, y (iii) prácticas de autoría que sean robustas frente a cambios de versión.

2. Conceptos esenciales

2.1. DAG, tarea y ejecución

- **DAG (Directed Acyclic Graph):** definición del workflow. Describe tareas y dependencias; no ejecuta por sí mismo.
- **Task:** unidad mínima de trabajo dentro del DAG. En Airflow, una *task* se materializa mediante un **Operator** (plantilla ejecutable) o mediante la **TaskFlow API** (`@task`).
- **DagRun:** una instancia de ejecución del DAG (por horario, manual o evento).
- **TaskInstance:** ejecución específica de una tarea dentro de un *DagRun*. Es lo que realmente "corre" y produce logs/estado.

En producción, lo importante es separar *definición* (DAG) de *instanciación* (DagRun) y de *ejecución* (TaskInstance). Esta separación explica por qué un DAG puede existir sin ejecutarse, y por qué un mismo DAG genera muchas ejecuciones históricas.

2.2. Operators, Sensors, Hooks y Providers

Airflow integra con sistemas externos a través de **providers** (paquetes versionados de forma independiente del core). Un provider aporta:

- **Operators:** realizan acciones (ejecutar SQL, disparar un job, invocar una API).
- **Sensors:** esperan condiciones (archivo disponible, partición creada). En versiones modernas, muchos sensores pueden ser *deferrable* (no consumen un worker durante la espera).
- **Hooks:** encapsulan conectividad (credenciales, clientes, sesiones) hacia el sistema externo.

En Airflow 3, varios operadores comunes se movieron a `apache-airflow-providers-standard` (por ejemplo `BashOperator` y `PythonOperator`), por lo que su importación típica se realiza desde `airflow.providers.standard...` [3], [8].

2.3. Comunicación y configuración: XComs, Variables y Connections

XCom es el mecanismo de comunicación entre tareas para pasar valores pequeños (metadatos, rutas, IDs). No está pensado para transportar dataframes o datasets completos. **Variable** y **Connection** representan configuración y credenciales administradas por Airflow.

En Airflow 3 se recomienda acceder a estos recursos mediante el **Task SDK** (`airflow.sdk`) o mediante el contexto de tarea [5].

3. Arquitectura de Airflow 3.x (qué corre y por qué importa)

Airflow 3.x introduce un cambio clave: **las tareas ya no deben acceder directamente a la base de metadatos**. En su lugar, el **API server** actúa como punto de acceso central para tareas y workers [7]. Esto reduce riesgos de seguridad (código de usuario con acceso total a metadatos) y mejora escalabilidad (menos conexiones directas al DB).

3.1. Componentes principales

Componente	Responsabilidad operativa
Scheduler	Decide qué *TaskInstances* están listas (dependencias satisfechas), aplica reglas de concurrencia, encola ejecución y coordina *backfills*. Airflow 3 incluye mejoras relevantes en *backfills* administrados por scheduler [2], [6].
DAG Processor	Proceso dedicado a parsear DAGs. Separa la "lectura de código" del resto del sistema, mejorando estabilidad; implica que el *código a nivel de módulo* debe ser liviano. [8]
API Server (UI/REST)	Expone la UI modernizada y la API REST. En Airflow 3, el comando `airflow api-server` reemplaza al webserver tradicional [8].
Executor / Workers	Ejecutan las tareas. Airflow soporta distintos ejecutores (Local, Celery, Kubernetes, Edge) y en Airflow 3 se refuerza una *Task Execution API* que permite mayor aislamiento y ejecución remota [6].
Metadata DB	Persistencia de estado (DAGs, runs, tasks, logs referenciados, conexiones/variables en algunos despliegues). En Airflow 3 se busca minimizar acceso directo desde tareas [7].

3.2. Implicaciones prácticas para autores de DAGs

1. **No asumir acceso a modelos internos** (SQLAlchemy, sesiones, tablas de Airflow). Está restringido [7].

2. **Usar la interfaz pública estable:** `airflow.sdk` para DAGs/tareas y utilidades públicas [5].

3. **Minimizar trabajo en tiempo de parseo:** evitar consultas, lectura de archivos grandes o llamadas de red en el nivel superior del archivo DAG.

4. Autoría de DAGs en Airflow 3: interfaz estable con `airflow.sdk`

4.1. Imports recomendados

Airflow 3 formaliza `airflow.sdk` como interfaz primaria para autores de DAGs: `dag()`, `task()`, `TaskGroup`, `Asset`, `Variable`, `Connection`, y acceso al contexto con `get_current_context()` [5].

Listing 1: Estructura mínima de un DAG con TaskFlow API (Airflow 3.x)

```python
from datetime import datetime
from airflow.sdk import dag, task

@dag(
    dag_id="ejemplo_minimo_taskflow",
    start_date=datetime(2025, 1, 1),
    schedule="0 2 * * *",
    catchup=False,
    tags=["demo", "airflow3"]
)
def pipeline():
    @task
    def paso_a():
        return "ok"

    @task
    def paso_b(msg: str):
        print(f"Mensaje: {msg}")

    paso_b(paso_a())

pipeline()
```

4.2. Acceso a Variables y Connections (sin DB directo)

Airflow 3 recomienda acceder a variables y conexiones vía **contexto** o directamente desde el SDK [5].

Listing 2: Acceso a Variable/Connection usando el contexto de tarea (recomendado)

```
from airflow.sdk import task, get_current_context

@task
def usa_configuracion():
    ctx = get_current_context()
    conn = ctx["conn"].get("mi_conn_id")   # Connection administrada por Airflow
    umbral = int(ctx["var"].value.get("umbral_calidad", "0"))  # Variable

    # Ejemplo: usar credenciales para conectar a un servicio externo
    print(conn.host, conn.login, umbral)
```

Listing 3: Acceso directo (SDK) a Connection/Variable

```
from airflow.sdk import Connection, Variable

mi_conn = Connection.get("mi_conn_id")
umbral = Variable.get("umbral_calidad", default_var="0")
```

4.3. Operadores del “standard provider”

En Airflow 3, para operadores comunes se requiere instalar `apache-airflow-providers-standard` y usar sus rutas de importación [1], [3], [8].

Listing 4: Ejemplo de uso de BashOperator desde el provider estándar

```
from datetime import datetime
from airflow.sdk import dag
from airflow.providers.standard.operators.bash import BashOperator

@dag(dag_id="ejemplo_bash", start_date=datetime(2025, 1, 1), schedule=None, catchup=False)
def ejemplo():
    BashOperator(
        task_id="listar",
        bash_command="ls -la"
    )

```

```
12 ejemplo()
```

5. Planificación: horarios, data intervals, catchup y backfill

5.1. Schedule vs. ejecución real

El atributo `schedule` define la intención (cron, @daily, o un *timetable*). El scheduler determina cuándo crear *DagRuns*. En Airflow 3, `catchup` es **False por defecto**, reduciendo ejecuciones masivas no deseadas al desplegar un DAG nuevo [8].

5.2. Backfills administrados por scheduler

El *backfill* es la re-ejecución histórica para intervalos pasados (por ejemplo, rehacer particiones faltantes). Airflow 3 mejora el soporte de backfills al administrarlos desde el scheduler, con monitoreo vía UI/API [2], [6]. Operativamente, esto facilita:

- control centralizado de concurrencia,
- diagnósticos consistentes,
- ejecución segura sin scripts manuales externos.

5.3. Versionado de DAG

Airflow 3 incorpora **DAG Versioning**: un *DagRun* se completa usando la versión del DAG con la que inició, incluso si se despliega una nueva versión a mitad de ejecución [2]. Esto reduce resultados inconsistentes por cambios de código durante un run activo, y mejora trazabilidad (código, estructura y logs asociados a una versión).

6. Control de flujo y confiabilidad (patrones operativos)

6.1. Reintentos, timeouts y reglas de disparo

A nivel de tarea, se recomienda definir parámetros como:

- `retries` y `retry_delay`: para fallas transitorias (red, rate limits).
- `execution_timeout`: evita tareas colgadas.
- `trigger_rule`: controla cómo se comporta una tarea cuando hay upstream fallando o saltado (*skipped*).

Estas configuraciones son parte del diseño de confiabilidad: un pipeline robusto asume fallas parciales y define qué significa "completo" de forma explícita.

6.2. Branching y ShortCircuit

Para rutas condicionales (por ejemplo, *si no hay nuevos datos, no cargar*), se usa branching (por ejemplo `BranchPythonOperator`) o `ShortCircuitOperator`. En Airflow 3 estos operadores están disponibles vía el provider estándar [1].

6.3. TaskGroup y legibilidad

Los DAGs crecen rápido. `TaskGroup` permite agrupar tareas y mantener legibilidad sin recurrir a SubDAGs (removidos) [5], [8].

6.4. Dynamic Task Mapping (map) como patrón de paralelismo

Cuando existe una lista de particiones/archivos, es común mapear una tarea sobre esa lista (p. ej., un task por archivo). El valor práctico es: paralelismo controlado + observabilidad por partición (cada mapeo es una TaskInstance con `map_index`).

7. Orquestación orientada a datos: Assets y scheduling por eventos

Airflow 3 refuerza un estilo *data-aware*: en lugar de pensar "ejecuta a las 2am", se puede pensar "ejecuta cuando el dataset X se actualice" [2], [6]. Este enfoque es útil cuando:

- los orígenes publican eventos (colas, almacenamiento, CDC),
- el horario fijo no garantiza disponibilidad de datos,
- se desea encadenar pipelines por producción/consumo de *assets*.

Listing 5: Ejemplo conceptual de asset como contrato de datos (Airflow 3.x)

```
from datetime import datetime
from airflow.sdk import dag, task, asset, Asset

ventas_diarias = Asset("warehouse://dw/fact_ventas_diarias")

@dag(dag_id="producer_ventas", start_date=datetime(2025, 1, 1),
   schedule="0 2 * * *", catchup=False)
def producer():
    @task(outlets=[ventas_diarias])
    def construir_fact():
        # construir partición y publicarla como asset
        pass
    construir_fact()

@dag(dag_id="consumer_kpis", start_date=datetime(2025, 1, 1),
   schedule=[ventas_diarias], catchup=False)
def consumer():
    @task
    def recalcular_kpis():
        pass
    recalcular_kpis()

producer()
consumer()
```

El ejemplo ilustra el contrato: el productor declara que *produce* un asset (`outlets`) y el consumidor se agenda cuando el asset está disponible (en lugar de por cron).

8. Observabilidad: UI, logs y trazabilidad

Airflow 3 moderniza la UI (React) y mejora la navegación entre vistas *task-centric* y *asset-centric* [2]. En operación, la observabilidad suele reducirse a tres preguntas:

1. **¿Qué falló y dónde?** (estado de TaskInstances, logs, stacktrace).

2. **¿Desde cuándo y con qué impacto?** (runs históricos, backfills, retrasos).

3. **¿Qué cambió?** (versionado de DAG, cambios de código, parámetros).

A nivel de diseño, conviene que cada tarea sea **idempotente** (re-ejecutable), con logs útiles y validaciones explícitas (por ejemplo conteos, checksums o pruebas de calidad).

9. Despliegue y operación

Aunque el despliegue suele ser responsabilidad del equipo de plataforma, el autor de DAG debe comprender restricciones básicas:

9.1. Ejecución local vs. producción

- **Local:** útil para aprender. Se puede usar el *Docker stack* oficial o un executor local.
- **Producción:** suele requerir múltiples componentes (scheduler, api-server, dag-processor, workers) y un DB robusto. El Helm Chart oficial es una vía común en Kubernetes [4].

9.2. CLI y "airflowctl"

Airflow 3 introduce `airflowctl` como CLI que interactúa exclusivamente con la API REST, evitando acceso directo a la base de metadatos y facilitando trazabilidad y auditoría [4].

9.3. Cambio de procesos de arranque

En Airflow 3, el webserver pasa a ser un API server genérico y el DAG processor se inicia por separado (aun en setups de desarrollo) [8]. Esto explica por qué "antes funcionaba con un solo comando" y ahora hay más procesos: es parte del rediseño arquitectónico.

10. Mini-caso: envolver un pipeline ETL existente con Airflow (sin redefinir ETL)

Asumiendo que ya existe un script o librería Python que hace (i) extraer, (ii) transformar y (iii) cargar, el valor de Airflow es **operar el ciclo completo** con control de errores y auditabilidad. Un patrón típico es:

1. **Preparación**: validar parámetros, verificar conectividad.
2. **Ejecución**: correr la carga (por partición, por fecha, por archivo).
3. **Calidad**: checks (conteos, nulos, umbrales).
4. **Publicación**: marcar un asset/listo, o notificar.

Listing 6: Esqueleto de DAG para orquestar un script ETL existente por fecha

```
from datetime import datetime
from airflow.sdk import dag, task, get_current_context

@dag(
    dag_id="orquesta_etl_por_fecha",
    start_date=datetime(2025, 1, 1),
    schedule="0 2 * * *",
    catchup=False,
    tags=["etl", "orquestacion"]
)
def orquesta():
    @task
    def parametros():
        ctx = get_current_context()
        # En Airflow, la fecha lógica del run sirve como partición
           de negocio
        logical_date = ctx["logical_date"]  # datetime
        return {"fecha": logical_date.strftime("%Y-%m-%d")}

    @task(retries=2)
    def ejecutar_etl(params: dict):
        fecha = params["fecha"]
```

```
22          # Llamar librería interna o script (ej: subprocess, import,
                etc.)
23          # run_pipeline(fecha=fecha)
24          print(f"Ejecutando ETL para {fecha}")
25
26      @task
27      def checks(params: dict):
28          fecha = params["fecha"]
29          # checks de calidad y auditoría: conteos, duplicados, nulos,
                etc.
30          print(f"Checks de calidad para {fecha}")
31
32      p = parametros()
33      ejecutar_etl(p) >> checks(p)
34
35  orquesta()
```

El ejemplo enfatiza una idea: **la partición (fecha lógica) vive en el run**, y las tareas son idempotentes y re-ejecutables. Esto habilita reintentos, backfills y diagnósticos de forma natural.

11. Buenas prácticas y anti-patrones

Buena práctica	**Anti-patrón típico**
Tareas idempotentes y con efectos controlados.	Tareas que “acumulan” datos sin control cuando se reintentan.
Usar `airflow.sdk` y providers; evitar imports internos.	Importar modelos internos o sesiones del metadata DB (bloqueado en Airflow 3).
XCom solo para metadatos (rutas, IDs, banderas).	Enviar grandes datasets por XCom.
Validar dependencias y credenciales como paso inicial.	Descubrir que falta una librería/permiso en el último paso del pipeline.
Minimizar trabajo en parseo del DAG.	Consultas o llamadas API a nivel global del archivo DAG.
Usar TaskGroup / mapping para legibilidad y escalabilidad.	DAGs monolíticos con decenas de tareas sin estructura.

12. Conclusiones

Airflow 3.x no es “otra herramienta ETL”, sino una plataforma para **operar workflows** con disciplina: planificación, dependencias, reintentos, trazabilidad y control de ejecuciones históricas. La versión 3 introduce cambios relevantes (API server, DAG processor independiente, Task Execution API y `airflow.sdk` como interfaz estable) que influyen directamente en cómo se escriben DAGs, cómo se accede a configuración (Variables/Connections) y cómo se mantiene seguridad del sistema [5], [6], [7].

Como siguiente paso práctico (en laboratorio), la recomendación es tomar un pipeline ya construido en Python, envolverlo en un DAG con particionamiento por fecha, agregar checks de calidad y practicar backfills controlados. Esta transición del “script que corre” al “proceso que se opera” es el objetivo central de aprender Airflow.

Referencias

[1] «airflow.providers.standard.operators.python — API Reference,» Apache Software Foundation, visitado 17 de feb. de 2026. dirección: https://airflow.apache.org/docs/apache-airflow-providers-standard/stable/_api/airflow/providers/standard/operators/python/index.html

[2] «Apache Airflow® 3 is Generally Available!» Apache Software Foundation, visitado 17 de feb. de 2026. dirección: https://airflow.apache.org/blog/airflow-three-point-oh-is-here/

[3] «BashOperator — apache-airflow-providers-standard Documentation,» Apache Software Foundation, visitado 17 de feb. de 2026. dirección: https://airflow.apache.org/docs/apache-airflow-providers-standard/stable/operators/bash.html

[4] «Documentation | Apache Airflow,» Apache Software Foundation, visitado 17 de feb. de 2026. dirección: https://airflow.apache.org/docs/

[5] «Public Interface for Airflow 3.0+ — Airflow Documentation,» Apache Software Foundation, visitado 17 de feb. de 2026. dirección: https://airflow.apache.org/docs/apache-airflow/stable/public-airflow-interface.html

[6] «Release Notes — Airflow Documentation (Airflow 3.0.0 Highlights),» Apache Software Foundation, visitado 17 de feb. de 2026. dirección: https://airflow.apache.org/docs/apache-airflow/stable/release_notes.html

[7] «Upgrading to Airflow 3 — Airflow Documentation,» Apache Software Foundation, visitado 17 de feb. de 2026. dirección: https://airflow.apache.org/docs/apache-airflow/stable/installation/upgrading_to_airflow3.html

[8] «Upgrading to Airflow 3 (Airflow 3.0.0) — Breaking Changes,» Apache Software Foundation, visitado 17 de feb. de 2026. dirección: https://airflow.apache.org/docs/apache-airflow/3.0.0/installation/upgrading_to_airflow3.html